

정세진 Backend Developer

문제를 분석하고 구조를 개선하는 백엔드 개발자

기능 구현을 넘어 문제를 분석하고, 상황에 맞는 구조를 설계하며 서비스를 개선하는 백엔드 개발자입니다.

GitHub	Email	블로그
https://github.com/sejinzx	jeongsejin789@gmail.com	https://vivid-reminiscences.tistory.com/

기술스택

Core

Java | Spring Boot | Spring Security | MySQL

Experienced

Redis | Apache Kafka | Docker | Kubernetes | GitHub Actions | NHN Cloud | React

Collaboration & Tools

GitHub | Notion | Slack

PROJECT 01

수강 신청 시스템

프로젝트 개요

수강 신청 시스템은 사용자가 강의를 조회하고 신청 및 관리할 수 있는 웹 서비스입니다.
Spring Boot 기반으로 인증/인가와 Kafka 기반 비동기 처리 구조를 적용하여 개발했습니다.

- 기간: 2026.05~06
- 형태: 개인 프로젝트
- 기술: Java, Spring Boot, Spring Security, JWT, MySQL, Redis, Apache Kafka, Docker, Kubernetes, GitHub Actions, JMeter

주요기능

- 회원가입 및 로그인
- 강의 조회
- 수강 신청/취소
- 신청 인원 제한 및 중복 신청 방지
- Kafka 기반 비동기 처리

시스템 아키텍처 및 개발 내용

- Spring Security + JWT 기반 인증/인가 및 Stateless 인증 구조 구현
- Redis TTL을 활용한 인증 데이터 만료 처리
- EntityGraph를 적용하여 N+1 문제 사전 방지
- Docker를 활용한 Kafka, MySQL, 애플리케이션 컨테이너 환경 구성
- 로컬 Kubernetes 환경 구축 및 GitHub Actions 기반 빌드·배포 자동화 파이프라인 구성

Kafka 기반 비동기 처리 구조 개선

- 비관적 락을 활용한 동시성 제어를 적용했지만, 요청이 증가하는 환경에서는 데이터베이스 락 경합이 발생할 수 있음을 확인했습니다.
- 이를 개선하기 위해 Kafka를 도입하여 수강 신청 요청을 메시지 큐에 적재하고, Producer가 요청 이벤트를 발행하면 Consumer가 이를 처리하는 비동기 구조를 구현했습니다. 이를 통해 요청 수신과 실제 수강 신청 처리를 분리했습니다.

구현 내용

- Kafka Producer를 통한 수강 신청 요청 이벤트 발행
- Kafka Consumer에서 신청 가능 여부 검증 및 수강 신청 처리
- Producer와 Consumer를 분리하여 요청 처리와 비즈니스 로직 분리

처리 흐름

Client → Producer → Kafka → Consumer → DB

검증

Docker 환경에서 JMeter를 활용해 최대 1,000건의 동시 요청 환경에서 테스트를 진행했습니다.

- 최대 1,000건의 동시 요청 환경에서 테스트 수행
- 정원 초과 신청 0건
- 중복 신청 0건
- 데이터 정합성 유지 확인

PROJECT 02

Talk-kit

프로젝트 개요

Talk-kit은 AI 기반 발표 연습 플랫폼으로, AI 대본 생성, 음성 녹음, 프로젝트 관리, 커뮤니티 피드백 기능을 제공하는 서비스입니다. MSA 기반으로 기능별 서비스를 분리하고, API Gateway를 통해 서비스 요청을 관리하는 구조에서 개발했습니다.

- 기간: 2024.06~10
- 인원: Backend 3 / Frontend 2
- 담당 역할: user-service, community-service 개발
- 기술: Java, Spring Boot, Spring Cloud Gateway, Eureka, MySQL, Redis, JWT, Docker, Kubernetes, NHN Cloud

담당 역할

user-service

- 사용자 CRUD API 구현
- Redis TTL 기반 이메일 인증 기능 구현
- 인증번호 저장 및 만료 처리 로직 개발

community-service

- 게시글 CRUD API 구현
- 게시글 좋아요 및 신고 기능 구현
- 게시글과 프로젝트 파일 통합 조회 API 구현

문제 해결 경험

문제

담당한 게시글 조회 API에서 project-service의 파일 정보를 함께 제공해야 했지만, 서비스별 데이터베이스가 분리되어 있어 직접 조회할 수 없었습니다.

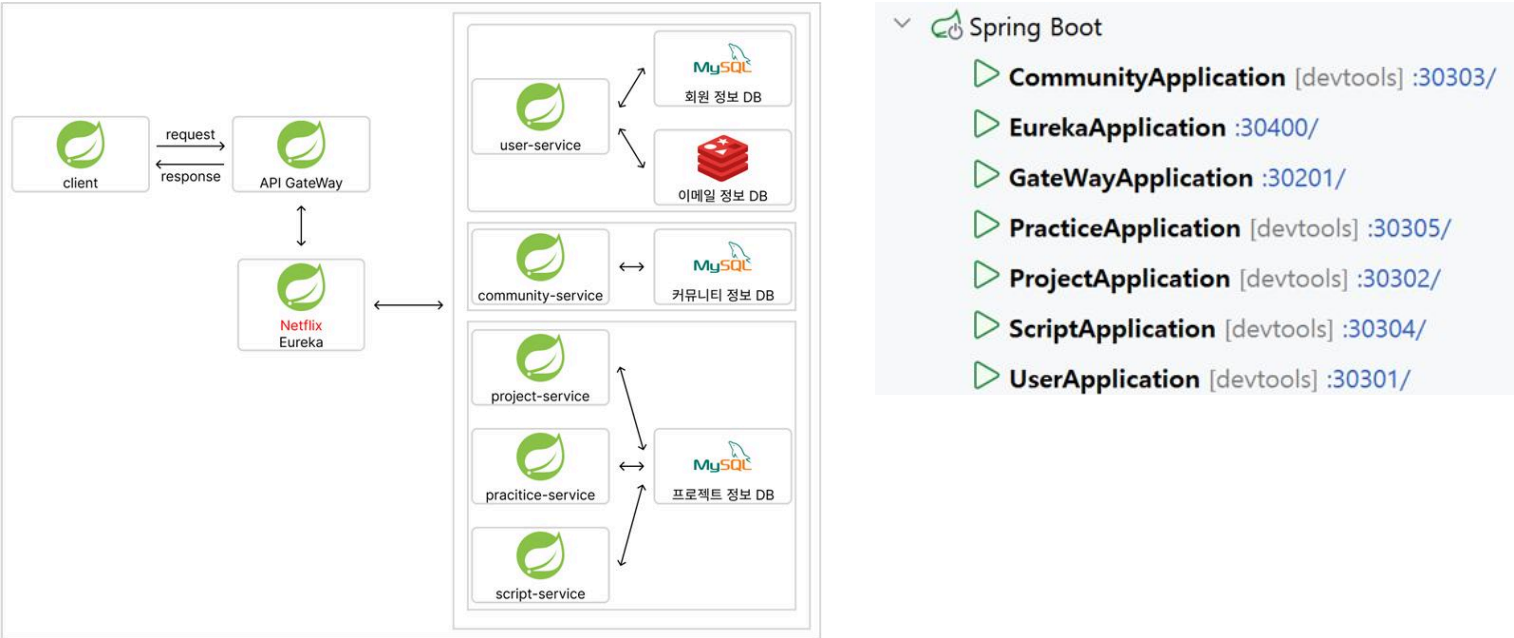
해결

Feign Client를 적용하여 project-service API를 호출하고, 게시글 정보와 파일 정보를 하나의 응답으로 조합하도록 구현했습니다.

결과

- 서비스 간 결합도를 유지하며 데이터 연동 구현
- 게시글과 프로젝트 파일 통합 조회 API 제공
- 화면 요구사항에 맞는 데이터 조회 구조 구현

시스템 아키텍처



PROJECT 03

일정 관리 시스템 (Scheduling)

프로젝트 개요

Scheduling은 일정 및 투두리스트를 관리할 수 있는 웹 애플리케이션입니다.
React와 Spring Boot를 기반으로 프론트엔드와 백엔드를 분리하고, 화면 요구사항에 맞는 REST API를 설계하여 개발했습니다.

- 기간: 2026.04~06
- 형태: 개인 프로젝트
- 기술: React, Java, Spring Boot, Spring Data JPA, MySQL, Docker

주요기능

- 일정 등록 / 수정 / 삭제
- 날짜 및 월 단위 일정 조회
- 투두리스트 관리
- REST API 기반 데이터 처리

개발 내용

- Spring Data JPA 기반 일정 및 투두 CRUD API 구현
- 날짜·월 단위 조회 API를 분리하여 화면 요구사항에 맞는 조회 구조 설계
- React와 REST API를 연동하여 일정 데이터 처리
- DTO를 활용해 화면에 필요한 데이터만 제공하도록 응답 구조 설계

캘린더 화면을 고려한 조회 API 재설계

문제

- 기존 API는 특정 날짜의 일정만 조회할 수 있어 월간 캘린더 화면을 구성하기 위해 날짜별 API를 반복 호출해야 했습니다.

```
GET /api/schedule/date?date=2026-06-10
```

해결

캘린더 화면에서 필요한 데이터를 한 번에 조회할 수 있도록 월 단위 조회 API를 설계했습니다.
Spring Data JPA를 활용해 해당 월의 일정을 한 번에 조회하고, 프론트엔드가 날짜별로 쉽게 활용할 수 있도록 응답 구조를 개선했습니다.

```
GET /api/schedule/month?year=2026&month=6
```

결과

- API 호출 횟수 최대 31회 → 1회 감소
- 프론트엔드 데이터 처리 로직 단순화
- 캘린더 화면에 최적화된 조회 API 및 응답 구조 구현